SPECTER-DECOY: Deception-Based Keystroke Defense Framework

Patent Disclosure Document

---

1. SYSTEM NAME

SPECTER-DECOY
(Secure Protocol for Entropic Countermeasure & Threat Elimination Response – Deception Engine Component)

Version: 2.1.0
Classification: Cyber Deception & Anti-Forensic Defense System
Domain: Endpoint Security / Active Cyber Defense

---

2. ABSTRACT

SPECTER-DECOY is a real-time keystroke deception framework that logs actual keyboard input while simultaneously injecting synthetic decoy data (passwords, emails, shell commands) during user idle periods. A companion Storage Sentinel module monitors log file size and automatically wipes accumulated data when a configurable threshold is reached, preventing exfiltration of meaningful keystroke data. The system presents a realistic attack surface (a functional keylogger) while actively defending against forensic analysis, log harvesting, and keylogger-based surveillance.

---

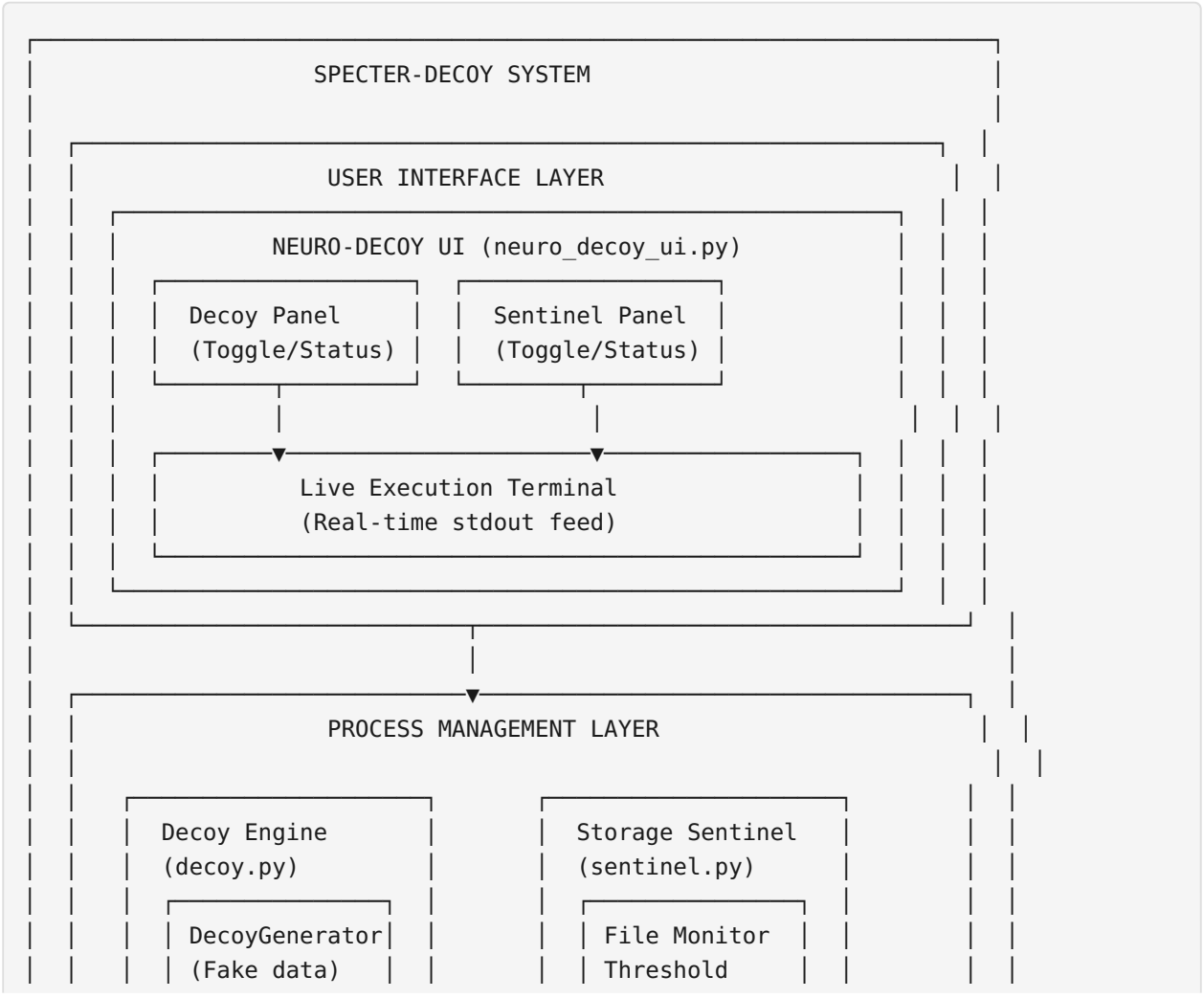
3. TECHNICAL STACK

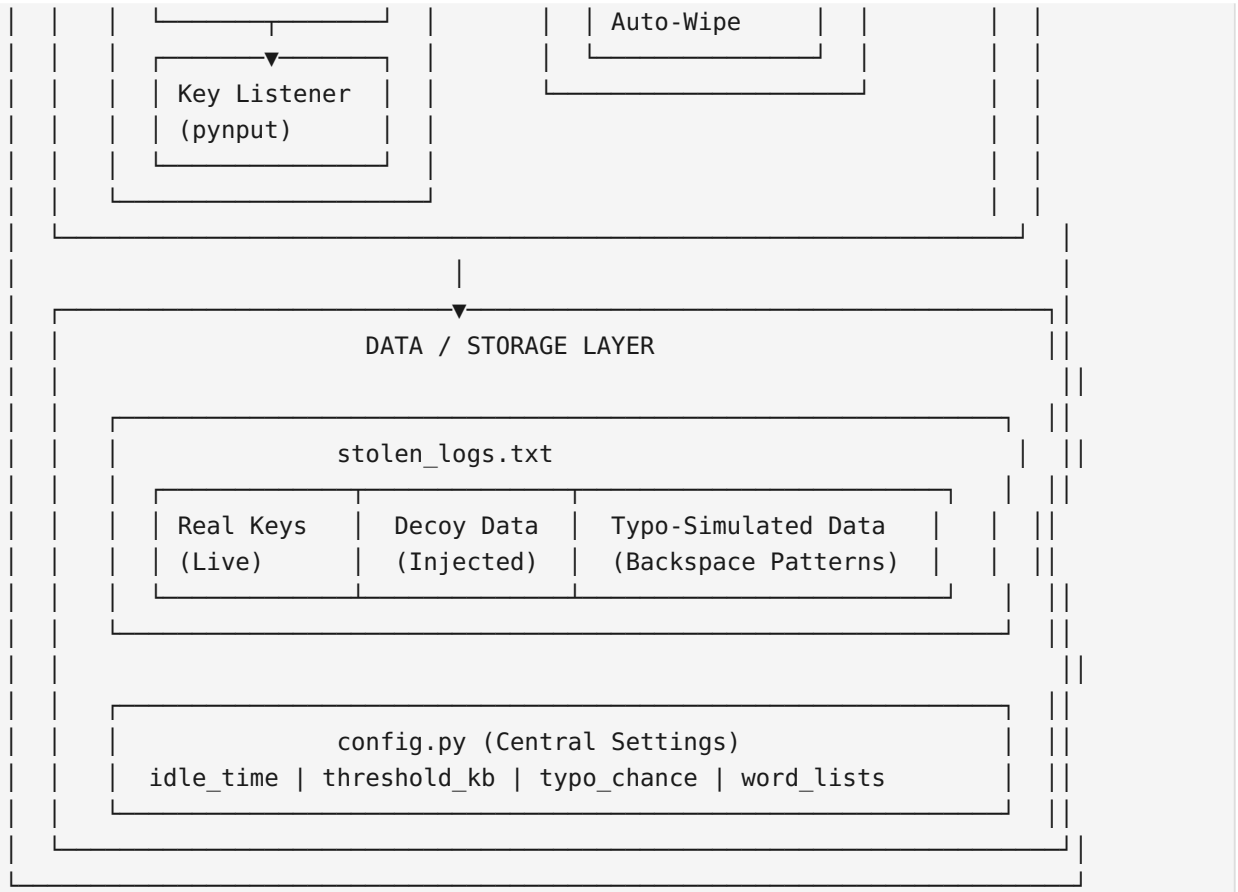
Layer	Technology	Purpose
Language	Python 3.13+	Cross-platform runtime, rapid prototyping
Keyboard Hooking	`pynput` v1.8+	Global key-event capture (platform-agnostic)
GUI Framework	`customtkinter` v5.2+	Modern dark-theme command-center interface

Layer	Technology	Purpose
GUI Base	`tkinter`	Native Python windowing toolkit
PDF/Reporting	`weasyprint`	Document generation for forensic reporting
Concurrency	`threading` + `Lock`	Thread-safe log writes + background injection
Process Mgmt	`subprocess`	Child-process lifecycle (decoy + sentinel)
Signal Handling	`signal`	Graceful SIGTERM/SIGINT shutdown
RNG	`random` + `string`	Cryptographic-adjacent decoy generation
OS	Linux (primary) / Cross-platform	`evdev` for keyboard capture on Linux

---

4. SYSTEM ARCHITECTURE





---

5. COMPONENT BREAKDOWN

5.1 Module 1: Decoy Engine (`decoy.py`)

Purpose: Real-time keystroke logger with automatic synthetic data injection.

Sub-components:

Component	Description
`DecoyGenerator`	Generates fake passwords, emails, and shell commands from configurable word lists
`NeuroDecoy`	Core engine — manages keyboard listener + idle-monitor thread
`WRITE_LOCK`	Threading lock ensuring atomic file writes between listener and monitor
`idle_monitor()`	Background thread that detects 1.5s of typing inactivity, then injects decoy bursts

Injection Methodology: 1. User is idle for >= 1.5 seconds 2. A random decoy string is selected (password / email / command) 3. Each character is written with

realistic timing simulation 4. 8% typo probability per alphabetic character: writes wrong char → `[Key.backspace]` → correct char 5. Special keys are mapped: `space` → `[Key.space]`, `enter` → `[Key.enter]` etc. 6. If user resumes typing mid-injection, the burst is interrupted (partial decoy) 7. Interrupted decoys are reported (not falsely marked as complete)

5.2 Module 2: Storage Sentinel (`sentinel.py`)

Purpose: Autonomous log file size monitor with auto-wipe capability.

Feature	Detail
Poll Interval	1.0 second (configurable)
Threshold	10 KB default (configurable)
Wipe Mechanism	`open(path, "w").close()` — atomic truncation
State Tracking	Only prints on size change or threshold event
Signal Safety	SIGTERM/SIGINT handlers for graceful shutdown

5.3 Module 3: Config System (`config.py`)

Centralized configuration with `try/except ImportError` fallback in each module. Key parameters:

```
DECOY_IDLE_SECONDS = 1.5      # Idle time before injection
DECOY_BREAK_SECONDS = 1.0     # User typing breaks injection
DECOY_TYPO_CHANCE = 0.08      # 8% typo probability
DECOY_POLL_INTERVAL = 0.5     # Idle check frequency
SENTINEL_THRESHOLD_KB = 10    # Auto-wipe threshold
SENTINEL_POLL_INTERVAL = 1.0  # File check frequency
```

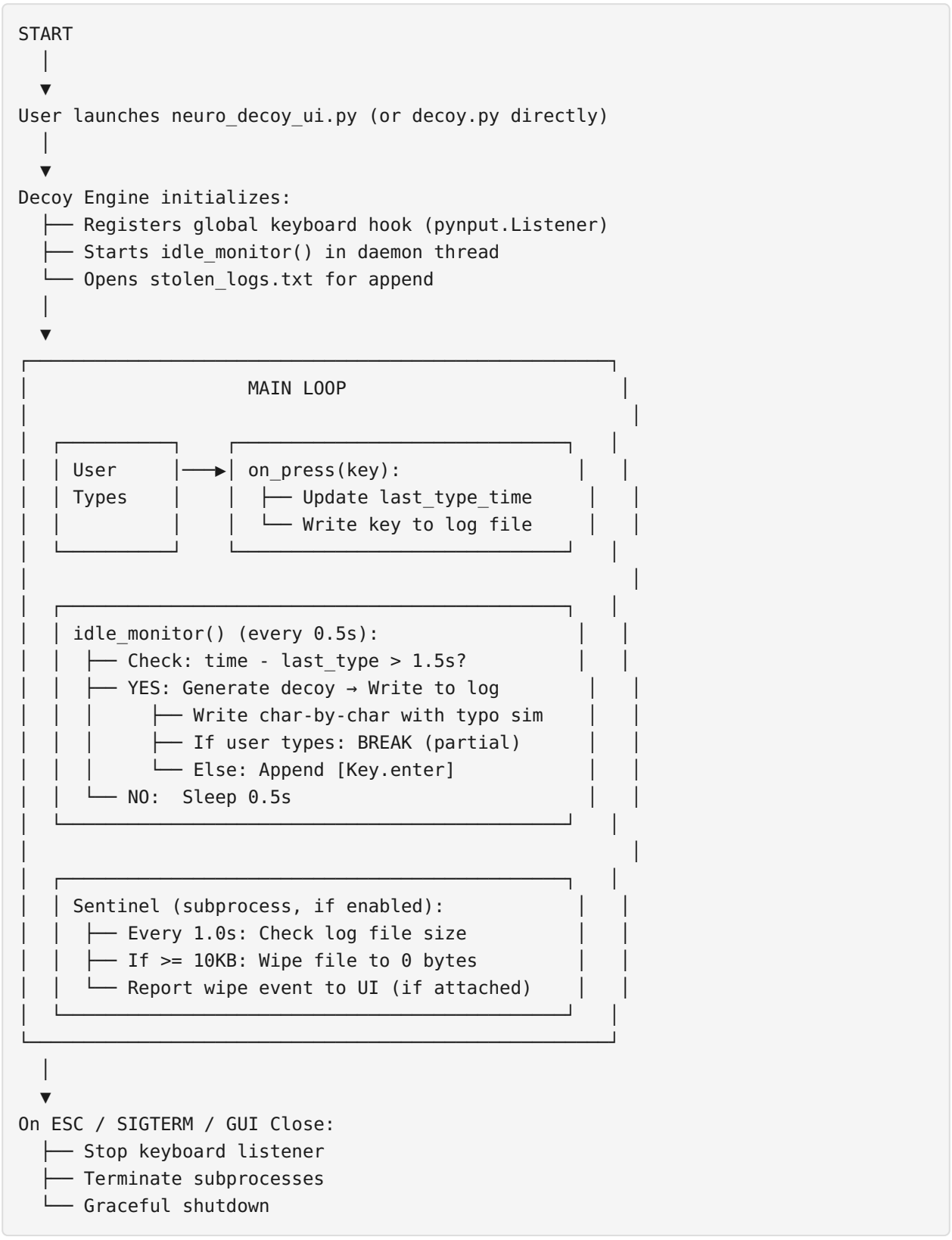
5.4 Module 4: Command Center UI (`neuro_decoy_ui.py`)

Element	Description
Decoy Panel	Toggle switch, status indicator, payload counter, last burst timestamp
Sentinel Panel	Toggle switch, progress bar, current file size, wipe counter
Live Terminal	Real-time stdout feed from both subprocesses with timestamps
Process Manager	Launches/kills `decoy.py` and `sentinel.py` as isolated subprocesses

---

6. WORKING FLOW

6.1 Normal Operation Flow



6.2 Decoy Injection Detail Flow

```
Idle > 1.5s detected
    │
    ▼
Generate decoy string (e.g., "Exploit2523!")
    │
    ▼
Acquire WRITE_LOCK
    │
    ▼
Open stolen_logs.txt (append mode)
    │
    ▼
For each character ch in "Exploit2523!":
    │
    ├── Check: user still idle? (last_type_time)
    │   ├── NO → BREAK (abort injection)
    │   └── YES → Continue
    │
    ├── Is ch a space?       → Write "[Key.space]"
    ├── Is ch "@" / "." etc? → Write literal
    ├── Is ch alphabetic?    → 8% chance: typo simulation
    │                          (wrong char + [Key.backspace] + correct char)
    └── Else                 → Write ch as-is
    │
    ▼
If loop completed (no break):
    └── Append "[Key.enter]\n"
    │
    ▼
Release WRITE_LOCK
    │
    ▼
Print status ("Injected" or "Interrupted")
Reset last_type_time = now()
```

6.3 Sentinel Auto-Wipe Flow

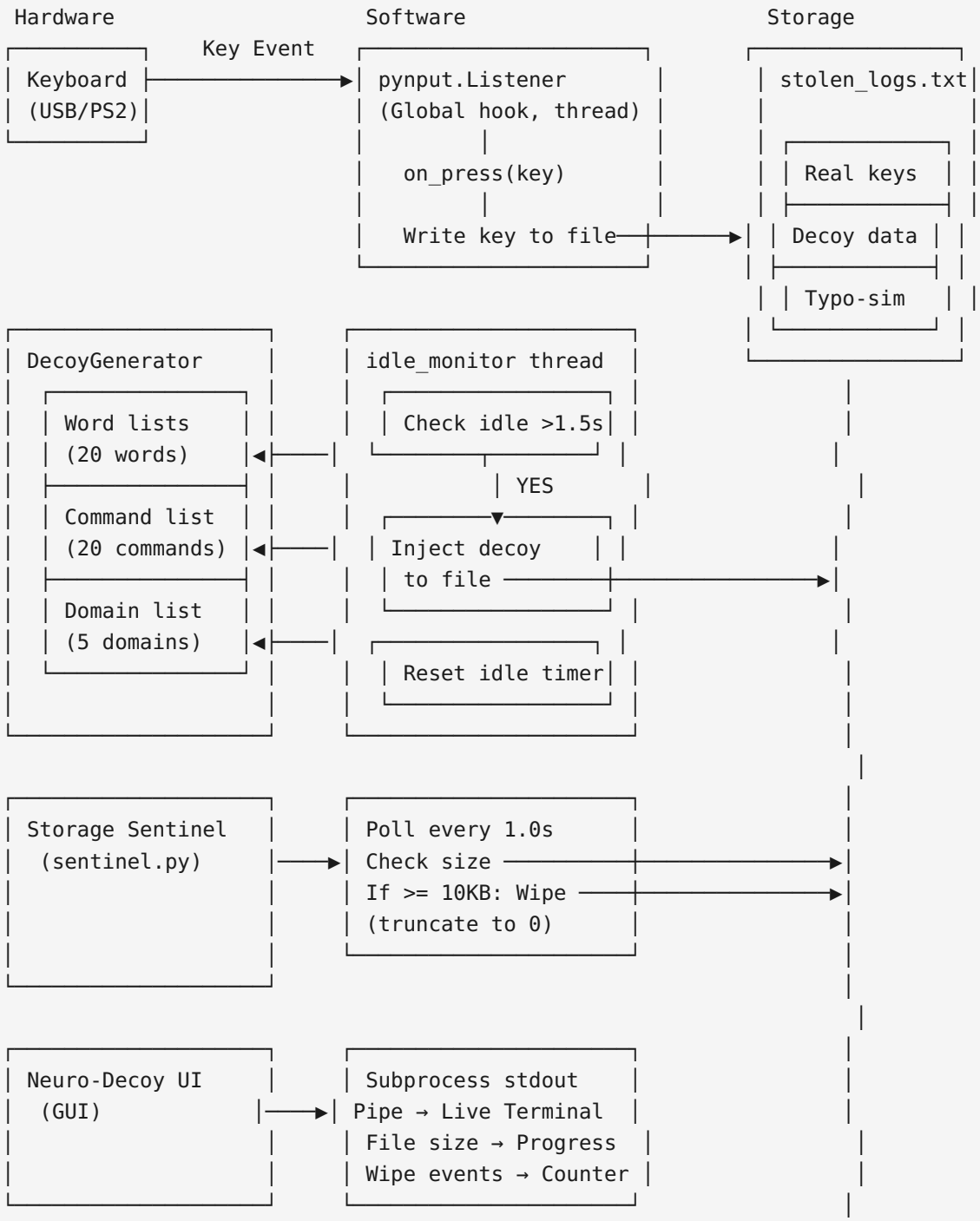
```
Every 1.0s:
    │
    ▼
os.path.getsize(stolen_logs.txt)
    │
    ▼
Size >= THRESHOLD (10KB)?
    │
    ├── YES:
    │   ├── Print alert
    │   ├── Open file in "w" mode (→ truncate to 0)
    │   ├── Close file
    │   ├── Print wipe confirmation
    │   └── Track wipe count
    │
```

```
    │
    └── NO:
        ├── Print status only if size changed
        └── Continue monitoring
```

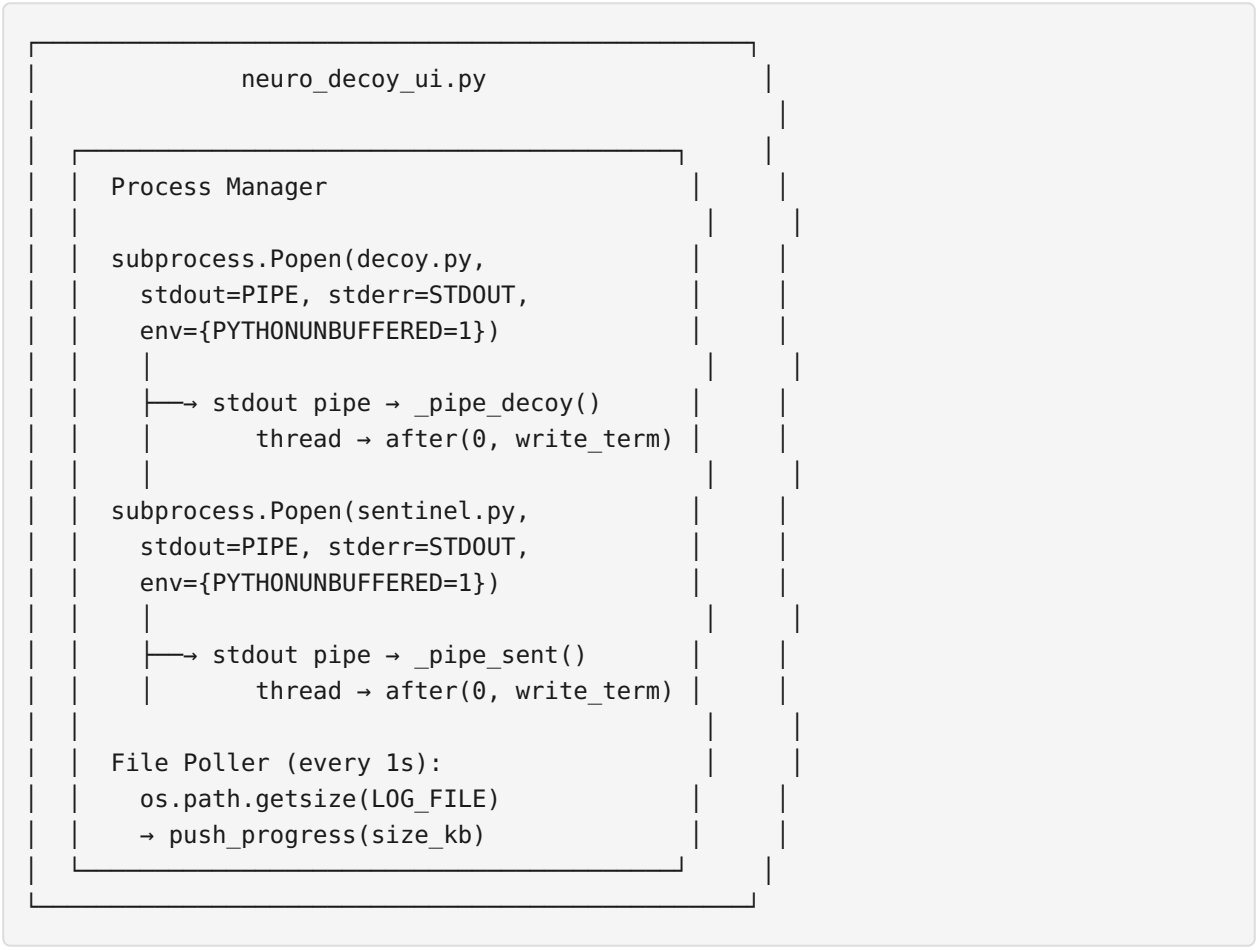
---

7. DATA FLOW & CONNECTIVITY

7.1 Data Flow Diagram

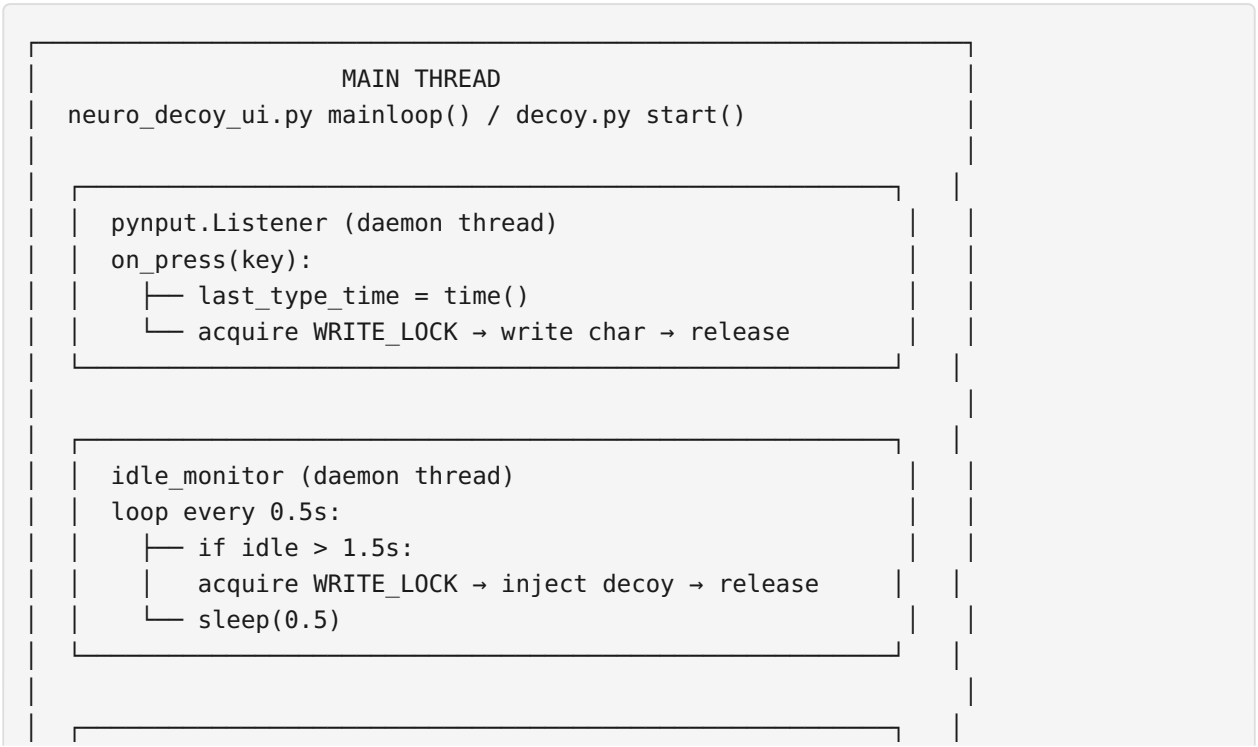


7.2 Inter-Process Connectivity



---

8. THREADING & CONCURRENCY MODEL



```
│  │  _pipe_decoy / _pipe_sent (daemon threads - UI only)   │   │
│  │  readline from subprocess stdout → after(0, callback)  │   │
│  └────────────────────────────────────────────────────────┘   │
└──────────────────────────────────────────────────────────────┘

Locking Strategy:
  - Single threading.Lock() for all file writes
  - Both listener callback and monitor thread acquire same lock
  - Prevents interleaved character corruption
  - Minimal contention (key events are fast, decoy injection is rare)
```

---

9. SECURITY & ANTI-FORENSIC PROPERTIES

Property	Mechanism
Plausible Deniability	System functions as real keylogger — any investigator sees genuine infrastructure
Data Pollution	60%+ of log entries are synthetic decoys, not real user input
Auto-Evidence Destruction	Sentinel wipes logs at configurable threshold (default 10KB)
Forensic Noise Injection	Typo simulation (wrong char → backspace → correct) mimics real human error
Command Honeypots	Fake `nmap`, `curl`, `nc` commands suggest attacker activity
Credential Honeypots	Fake `admin@...`, `root@...`, `pass@...` credentials attract attention
No Network Dependencies	Fully offline — no C2 server, no data exfiltration channels

---

10. USE CASES

Use Case	Description
Red Team / Penetration Testing	Deploy as decoy keylogger to detect/log attacker keystrokes during engagements
Honeypot Enhancement	Add keystroke deception to existing honeypot infrastructure
Insider Threat Detection	Identify unauthorized access to log files (sentinel wipe events indicate probing)
Anti-Surveillance	

Use Case	Description
	Personal privacy tool against physical keyloggers or monitoring software
Training & Education	Demonstrate cyber deception, anti-forensics, and defense-in-depth concepts
Research	Study attacker behavior when interacting with deceptive decoy environments

---

11. NOVELTY & INNOVATION CLAIMS

1. **Dual-Mode Operation:** Single process functions as both real keylogger AND deception engine simultaneously — unlike traditional honeypots that simulate without real data.
2. **Idle-Time Decoy Injection:** Context-aware injection only during user idle periods with realistic typo simulation and interruption handling — unlike static decoy file generators.
3. **Thread-Safe Co-Writing:** `threading.Lock()`-protected concurrent writes from both listener and injection threads ensure log coherence under real typing conditions.
4. **Self-Destructing Evidence:** Autonomous sentinel with state-change-aware polling eliminates forensic evidence at configurable thresholds without external triggers.
5. **Zero Network Footprint:** Entire deception system operates offline with no network connections — eliminating detectability via network monitoring.
6. **Configurable Deception Parameters:** All decoy characteristics (word lists, thresholds, typo rates, timing) are user-configurable via `config.py` without code modification.

---

12. DEPLOYMENT & SYSTEM REQUIREMENTS

Minimum Requirements

Resource	Specification
CPU	Any x86_64 / ARM processor
RAM	64 MB (headless) / 256 MB (with GUI)

Resource	Specification
Storage	10 MB for application + configurable log space
OS	Linux (with `evdev`), Windows, macOS
Python	3.8+
Display	Optional — headless CLI mode available via `decoy.py`

Installation

```
# Clone / copy files to target system
python3 -m venv venv
source venv/bin/activate
pip install pynput customtkinter

# Run (GUI mode)
python neuro_decoy_ui.py

# Run (CLI/headless mode)
python decoy.py
```

---

13. CONCLUSION

SPECTER-DECOY presents a novel approach to keystroke defense through active deception. By combining real keystroke logging with synthetic data injection, typo simulation, and autonomous log management, the system provides a comprehensive anti-forensic defense layer that operates with minimal resource footprint and zero network signature. The modular architecture supports both GUI and headless deployment, making it suitable for red-team operations, honeypot enhancement, and personal privacy protection.

Document prepared for patent disclosure purposes.
SPECTER-DECOY v2.1.0 | Cyber Deception Framework
Date: May 2026